

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO**INGENIERÍA EN SOFTWARE****Asignatura:** Aplicaciones Distribuidas (Séptimo Semestre “A”)

**Análisis Comparativo Profundo de Estilos
Arquitectónicos Distribuidos: Microservicios vs.
Arquitectura Dirigida por Eventos (EDA)***Evaluación Crítica Aplicada al Sistema de Gestión de Solicitudes para Soporte
Técnico de Internet*

Autor: Carpio Mendoza Carlos José — **Docente:** Ing. Gleiston Cicerón Guerrero Ulloa, Mgs.**Fecha:** Mayo 2026 — Quevedo – Ecuador**Resumen**

El presente análisis comparativo aborda la evaluación crítica de dos de los estilos arquitectónicos distribuidos más prevalentes en la ingeniería de software contemporánea: la Arquitectura de Microservicios orientada a la comunicación síncrona (Request-Response) y la Arquitectura Dirigida por Eventos (EDA, Event-Driven Architecture)[cite: 13]. La investigación se contextualiza y aplica de manera directa sobre el dominio del *Sistema de Gestión de Solicitudes para Soporte Técnico de Internet*, un ecosistema distribuido diseñado para mitigar la degradación operativa de los canales informales de soporte en organizaciones medianas y pequeñas[cite: 14]. A través de un análisis fundamentado en atributos de calidad, el Teorema de CAP y patrones de consistencia distributiva, se demuestran las limitaciones del acoplamiento temporal inherente a las peticiones HTTP/gRPC tradicionales frente a la resiliencia y el procesamiento asíncrono no bloqueante provisto por un esquema basado en eventos intermediados por un *Message Broker*[cite: 15]. Se concluye que la hibridación estratégica de ambos estilos optimiza la tolerancia a fallos, la escalabilidad elástica y la autonomía de los componentes de negocio fundamentales del sistema analizado[cite: 16].

Palabras clave: Sistemas Distribuidos, Microservicios, EDA, Teorema de CAP, Consistencia Eventual, Soporte Técnico[cite: 17].

1. Objetivo del Análisis Técnico

El presente documento tiene como objetivo evaluar críticamente, mediante un enfoque formal y de atributos de calidad, las compensaciones (*trade-offs*) de diseño arquitectónico entre los modelos síncronos Petición-Respuesta y los esquemas asíncronos Dirigidos por Eventos

(EDA). Se busca fundamentar teórica y matemáticamente la reestructuración del *Sistema de Gestión de Solicitudes para Soporte Técnico de Internet* hacia un ecosistema basado en microservicios desacoplados para mitigar el agotamiento de hilos, optimizar la tolerancia a fallos y garantizar la disponibilidad elástica de la plataforma bajo condiciones extremas de contención de red.

Índice

1. Objetivo del Análisis Técnico	1
2. Introducción	3
3. Descripción y Modelado del Dominio del Proyecto Integrador	4
3.1. Componentes Nucleares del Ecosistema Distribuidos	4
4. Fundamentos Teóricos de los Estilos Arquitectónicos Distribuidos	6
4.1. Microservicios Síncronos Basados en Petición-Respuesta	6
4.2. Arquitectura Dirigida por Eventos (EDA)	6
5. Análisis Comparativo Profundo y Evaluación de Atributos de Calidad	8
5.1. Comportamiento bajo el Teorema de CAP	8
5.2. Matriz de Evaluación de Atributos de Calidad	8
6. Aplicación Práctica y Modelado de Escenarios en el Sistema de Soporte	9
6.1. Escenario Síncrono: Flujo Crítico Bloqueante	9
6.2. Escenario Asíncrono (EDA): Coreografía de Eventos no Bloqueante	10
7. Compensaciones (Trade-offs) y Desafíos de Implementación	11
7.1. Consistencia de Datos Distribuida: El Patrón Saga	11
7.2. Complejidad Operativa y Trazabilidad Distribuida	11
8. Conclusiones	12
9. Referencias Bibliográficas	13

Índice de tablas

1. Matriz comparativa de atributos de calidad entre Microservicios Síncronos y EDA.	9
---	---

2. Introducción

El diseño de sistemas de software a gran escala ha experimentado una transformación fundamental en la última década, alejándose de los patrones estructurales monolíticos hacia los paradigmas de sistemas distribuidos. Esta transición no responde a una corriente efímera en la industria de la ingeniería de software, sino a la necesidad tecnológica imperativa de satisfacer atributos de calidad altamente exigentes, tales como la disponibilidad global, la escalabilidad horizontal automatizada y la tolerancia a particiones físicas de la infraestructura de red.

En el ámbito específico de los sistemas distribuidos, el diseño arquitectónico dicta la manera en que múltiples procesos computacionales independientes cooperan, intercambian datos y coordinan sus estados de negocio internos para simular un sistema único y coherentemente estructurado de cara al usuario final. Sin embargo, la construcción de este tipo de sistemas introduce una capa sustancial de complejidad operativa y conceptual. Los ingenieros de software deben diseñar bajo la premisa de las denominadas *Falacias de la Computación Distribuida*, formuladas originalmente por L. Peter Deutsch. Asumir de forma ingenua que la red es siempre confiable, que la latencia de transporte de paquetes es nula, que el ancho de banda disponible es infinito o que la topología de la infraestructura es estática y totalmente segura, conduce invariablemente a fallas catastróficas del sistema cuando este se despliega en entornos de producción reales.

En consecuencia, la selección del estilo de comunicación y la topología de acoplamiento entre los componentes distribuidos constituye la decisión más crítica en las fases iniciales del ciclo de vida del software. El presente análisis comparativo examina a profundidad dos filosofías arquitectónicas contrapuestas pero complementarias: la Arquitectura de Microservicios estructurada mediante el patrón de comunicación síncrona Petición-Respuesta (*Request-Response*), y la Arquitectura Dirigida por Eventos (*Event-Driven Architecture*, EDA). Este contraste se realiza con un enfoque estrictamente aplicado al *Sistema de Gestión de Solicitudes para Soporte Técnico de Internet*. A lo largo del documento, se argumentará conceptualmente cómo la adopción de esquemas asíncronos desacoplados mitiga la propagación de fallas en cascada y garantiza la continuidad del negocio técnico en escenarios de alta contención.

3. Descripción y Modelado del Dominio del Proyecto Integrador

La provisión ininterrumpida y eficiente del servicio de conectividad a Internet se ha consolidado como un recurso crítico e indispensable para la operatividad diaria de las organizaciones modernas. A pesar de esta dependencia tecnológica, una fracción considerable de pequeñas y medianas empresas continúan gestionando los incidentes, fallas y requerimientos de soporte técnico a través de canales de comunicación informales, tales como aplicaciones de mensajería instantánea, llamadas telefónicas directas o hilos de correo electrónico sin centralizar. Esta carencia de infraestructura formal da origen a una degradación sistemática de la calidad del servicio, caracterizada por la pérdida recurrente de la trazabilidad de los tickets, la incapacidad de establecer prioridades de atención basadas en el impacto operativo, la asimetría severa de información entre el cuerpo técnico y los usuarios, y una prolongación inaceptable en los tiempos de resolución de incidencias.

Para revertir este panorama, se define el *Sistema de Gestión de Solicitudes para Soporte Técnico de Internet*. El objetivo principal de este sistema es estructurar, automatizar y auditar el ciclo de vida completo de un ticket de soporte, abarcando desde su registro inicial por el usuario afectado hasta su diagnóstico automático, notificación al personal técnico y posterior cierre formal.

Si esta aplicación se concibiera bajo un esquema monolítico centralizado tradicional, la infraestructura entera heredaría un *Single Point of Failure* (SPOF) o Punto Único de Fallo. Bajo este escenario desfavorable, si el componente encargado de procesar las notificaciones o la base de datos relacional central experimenta una condición de bloqueo o sobrecarga, toda la plataforma colapsaría por completo, impidiendo de este modo que los usuarios puedan reportar fallas críticas de conectividad. Para evitar este riesgo sistémico, se propone una arquitectura distribuida que segmenta las capacidades de negocio en componentes independientes que interactúan a través de interfaces de red bien definidas.

3.1. Componentes Nucleares del Ecosistema Distribuidos

El dominio del sistema de soporte se encuentra descompuesto en cuatro componentes fundamentales que requieren un nivel estricto de aislamiento funcional y de datos:

1. **Componente de Autenticación y Autorización:** Este módulo es el encargado exclusivo de la gestión integral de identidades organizacionales, la verificación de credenciales de acceso y la implementación del modelo de Control de Acceso Basado en Roles (RBAC). Su salida principal es la generación de tokens criptográficos firmados (JSON Web Tokens) que aseguran las peticiones interservicios.
2. **Componente de Gestión de Reportes e Incidencias:** Representa el núcleo transaccional del dominio. Gobierna los estados mutables de las solicitudes de soporte (tales como *Abierto*, *En Proceso*, *Escalado* y *Resuelto*) y persiste la información estructural del ticket de conectividad en almacenes de datos optimizados para alta concurrencia escritural.

3. **Componente de Notificaciones Automatizadas:** Actúa como el motor de despacho de alertas del ecosistema. Su responsabilidad radica en procesar los avisos destinados al personal técnico y a los administradores a través de protocolos de correo electrónico o pasarelas de mensajería en tiempo real, asegurando que los tiempos de asignación se mantengan óptimos.
4. **Componente de Inteligencia Artificial (IA):** Este servicio especializado expone capacidades de procesamiento de lenguaje natural. Su función analítica es interceptar la descripción textual del problema provista por el usuario y extraer variables críticas para preclasificar y categorizar el tipo de incidente de conectividad (por ejemplo, fallas en capa física, problemas de enrutamiento o saturación de ancho de banda) sin requerir intervención humana preliminar.

4. Fundamentos Teóricos de los Estilos Arquitectónicos Distribuidos

Para evaluar los mecanismos de interacción dentro del sistema de soporte, es indispensable formalizar las bases teóricas de los dos estilos arquitectónicos en cuestión, prestando especial atención a sus topologías de acoplamiento.

4.1. Microservicios Síncronos Basados en Petición-Respuesta

La arquitectura clásica de microservicios promueve la descomposición de una aplicación en servicios pequeños y autónomos, enfocados en un contexto delimitado (*Bounded Context*). En su variante más común, la comunicación interservicio se modela de forma síncrona mediante el patrón Petición-Respuesta (*Request-Response*), implementado típicamente sobre los protocolos HTTP/REST o gRPC.

El comportamiento del ecosistema distribuido bajo fallos de infraestructura se rige de forma estricta por el Teorema PACELC (extensión directa del Teorema de CAP). Si existe una partición de red (P), el sistema debe optar entre Disponibilidad (A) o Consistencia (C); de lo contrario (E), se analiza el balance analítico entre Latencia (L) y Consistencia (C). La disponibilidad individual (A_i) de un microservicio transaccional síncrono se modela formalmente a través de sus métricas de confiabilidad:

$$A_i = \frac{MTBF_i}{MTBF_i + MTTR_i} \quad (1)$$

Donde MTBF representa el tiempo medio entre fallos y MTTR el tiempo medio de reparación. Para un grafo de dependencias donde n microservicios interconectados de forma síncrona se invocan secuencialmente en serie, la disponibilidad global del sistema decae de forma multiplicativa según la probabilidad conjunta de supervivencia:

$$A_{sistema} = \prod_{i=1}^n A_i \quad (2)$$

Esta inecuación demuestra que la disponibilidad del *Componente de Gestión de Reportes* se degrada exponencialmente a medida que se añaden llamadas síncronas bloqueantes al hilo de ejecución ($A_{sistema} < \min(A_i)$). Al implementar una Arquitectura Dirigida por Eventos (EDA), el acoplamiento temporal se rompe debido a la intermediación de buffers inmutables en el *Message Broker*, aislando la probabilidad de fallo y transicionando el sistema hacia un esquema de alta disponibilidad tolerante a particiones (AP/EL) bajo el modelo de consistencia eventual BASE.

4.2. Arquitectura Dirigida por Eventos (EDA)

La Arquitectura Dirigida por Eventos rompe el acoplamiento temporal y espacial mediante la introducción de un intermediario centralizado de mensajería, denominado *Event Broker* o *Message Broker* (como Apache Kafka o RabbitMQ). En este paradigma, los componentes distribuidos no invocan comandos directos sobre otros servicios; en su lugar, reaccionan ante

la publicación de *Eventos*, los cuales se definen formalmente como registros inmutables de un hecho significativo ocurrido en el pasado del dominio de negocio.

El flujo operativo se divide en tres actores: Productores, el Broker y Consumidores. El productor emite un evento al Broker de manera asíncrona y continúa su ejecución de inmediato sin esperar confirmación o procesamiento por parte de terceros. Los consumidores interesados se suscriben a canales o tópicos específicos dentro del Broker para procesar los eventos a su propio ritmo (*Backpressure*). Al no existir comunicación directa entre componentes, EDA maximiza el aislamiento, permitiendo que los servicios operen en un estado de completo desconocimiento mutuo, lo que incrementa sustancialmente la resiliencia global ante caídas parciales de la infraestructura.

5. Análisis Comparativo Profundo y Evaluación de Atributos de Calidad

La selección entre microservicios síncronos y arquitecturas dirigidas por eventos no debe responder a preferencias dogmáticas, sino a una evaluación matemática y técnica rigurosa de las compensaciones involucradas.

5.1. Comportamiento bajo el Teorema de CAP

El Teorema de CAP, postulado por Eric Brewer, determina que un sistema distribuido operando sobre una red propensa a particiones físicas (P) solo puede garantizar simultáneamente dos de los siguientes tres atributos: Consistencia Inmediata (C) o Disponibilidad General (A).

Los microservicios síncronos tradicionales tienden a priorizar esquemas de tipo CP. Cuando el Componente de Gestión de Reportes realiza una llamada síncrona HTTP hacia el Componente de Inteligencia Artificial para categorizar un ticket, el sistema busca mantener la consistencia inmediata de los datos del reporte. Si la red se particiona o el servicio de IA no está disponible, la transacción síncrona falla y la solicitud del usuario es rechazada. Se sacrifica la disponibilidad global del sistema en pos de asegurar que no existan inconsistencias temporales en el estado del registro.

Por el contrario, la Arquitectura Dirigida por Eventos se alinea de forma nativa con los sistemas de tipo AP, fundamentados en el modelo de consistencia BASE (*Basically Available, Soft state, Eventual consistency*). Bajo este enfoque, cuando se registra una incidencia, el Componente de Reportes persiste el estado inicial en su base de datos local y publica inmediatamente un evento denominado `TicketCreado`. El sistema se mantiene completamente disponible para el usuario. Aunque el Componente de IA o el de Notificaciones se encuentren desconectados temporalmente debido a un fallo en la red, el Broker retiene el mensaje de forma persistente. Una vez restablecida la conectividad, los consumidores procesan los mensajes pendientes, alcanzando la consistencia de datos de manera eventual.

5.2. Matriz de Evaluación de Atributos de Calidad

Para sintetizar las divergencias operativas entre ambos enfoques arquitectónicos, la Tabla 1 presenta una matriz detallada que contrasta los criterios de ingeniería de software más críticos en el contexto de sistemas altamente distribuidos.

Tabla 1: Matriz comparativa de atributos de calidad entre Microservicios Síncronos y EDA.

Atributo de Calidad	Microservicios Síncronos	Arquitectura Dirigida por Eventos
Acoplamiento Temporal	Alto (Ambos servicios deben estar en línea simultáneamente).	Nulo (El productor y el consumidor operan de forma diferida).
Acoplamiento Espacial	Medio (Requiere mecanismos de <i>Service Discovery</i> o DNS).	Nulo (Los servicios solo conocen la interfaz del <i>Event Broker</i>).
Manejo de Carga Fluente	Deficiente (Picos de carga pueden saturar los hilos del servidor).	Excelente (Implementa <i>Backpressure</i> nativo mediante colas).
Consistencia de Datos	Inmediata (Garantizada mediante transacciones locales inmediatas).	Eventual (Garantizada a través de propagación asíncrona).
Complejidad Operativa	Baja-Media (Flujos secuenciales fáciles de depurar).	Alta (Requiere herramientas de observabilidad avanzadas).

6. Aplicación Práctica y Modelado de Escenarios en el Sistema de Soporte

Para validar de forma empírica los fundamentos expuestos, se modela el flujo crítico del sistema de soporte técnico cuando un cliente de la organización reporta una pérdida total del servicio de Internet mediante la interfaz web.

6.1. Escenario Síncrono: Flujo Crítico Bloqueante

En un esquema clásico de microservicios síncronos, el proceso se ejecuta de la siguiente manera:

1. El usuario envía una petición HTTP POST al *Componente de Gestión de Reportes e Incidencias*.
2. El servicio de Reportes abre un hilo de ejecución, genera el ID del ticket y, antes de responder al cliente, realiza una llamada RPC síncrona hacia el *Componente de Inteligencia Artificial* para categorizar el problema.
3. El servicio de IA procesa la solicitud lingüística durante un lapso de 2.5 segundos.
4. Una vez obtenida la categoría, el servicio de Reportes efectúa una segunda llamada síncrona hacia el *Componente de Notificaciones* para despachar la alerta al técnico asignado.
5. El servicio de Notificaciones interactúa con el servidor SMTP externo, añadiendo otra latencia de 1.8 segundos.

Bajo esta topología síncrona, el usuario experimenta un tiempo de espera de red superior a los 4.3 segundos. Si en una hora pico se acumulan múltiples solicitudes concurrentes, el pool de hilos del servicio de Reportes se agota por completo esperando las respuestas de la IA y de Notificaciones. Si el módulo de IA se cae por mantenimiento, el flujo entero falla, impidiendo que el ticket de soporte se registre en la base de datos.

6.2. Escenario Asíncrono (EDA): Coreografía de Eventos no Bloqueante

Al transformar el ecosistema en una arquitectura dirigida por eventos, el proceso se optimiza radicalmente mediante una topología de coreografía. El *Componente de Gestión de Reportes* recibe la solicitud, persiste el ticket de soporte inmediatamente en su almacén de datos local con estado "PENDIENTE_ANALISIS" y publica de forma asíncrona el evento `TicketCreadoEvent` hacia el *Event Broker* en un tiempo récord de 15 milisegundos, retornando de inmediato una respuesta de éxito al usuario.

El payload estructurado del evento adopta un formato estandarizado en JSON, permitiendo que cualquier componente interesado consuma los datos de forma agnóstica:

```
{
  "eventId": "evt_987654321_2026",
  "eventType": "ec.edu.uteq.soporte.ticket.created",
  "timestamp": "2026-06-03T20:40:00Z",
  "data": {
    "ticketId": "tck_2026_0012",
    "clienteId": "usr_carlos_carpio",
    "descripcion": "Caída total de fibra optica en el nodo central",
    "estadoInicial": "PENDIENTE_ANALISIS"
  }
}
```

A partir de este instante, el *Componente de Inteligencia Artificial* y el *Componente de Notificaciones Automatizadas*, que actúan como consumidores independientes suscritos al tópico de tickets, detectan el evento en el Broker. El servicio de IA procesa el texto a su propio ritmo y, al finalizar, publica un nuevo evento denominado `TicketCategorizadoEvent`. Por su parte, el servicio de Notificaciones extrae los datos del mensaje y envía los avisos al personal técnico sin haber bloqueado jamás la navegación del cliente en Quevedo. Si el servicio de IA llega a desconectarse, los mensajes permanecen seguros en la cola del Broker, garantizando que el sistema nunca pierda un solo reporte de soporte técnico.

7. Compensaciones (Trade-offs) y Desafíos de Implementación

A pesar de las ventajas evidentes en cuanto a resiliencia y aislamiento, la adopción de una arquitectura plenamente dirigida por eventos introduce serios desafíos de ingeniería que el equipo de desarrollo debe mitigar críticamente.

7.1. Consistencia de Datos Distribuida: El Patrón Saga

En sistemas distribuidos orientados a eventos, las transacciones ACID tradicionales de las bases de datos relacionales centralizadas dejan de ser viables debido a la ausencia de un coordinador transaccional único. Para asegurar la integridad de los datos a través de múltiples microservicios sin reintroducir acoplamiento síncrono, se vuelve obligatorio implementar el **Patrón Saga**.

En lugar de ejecutar una sola transacción global bloqueante, una Saga coordina una secuencia de transacciones locales e independientes. Cada servicio ejecuta su transacción local y publica un evento que dispara la siguiente fase de la Saga. Si una de las transacciones locales falla (por ejemplo, si el componente de IA determina que el ticket contiene datos fraudulentos o no autorizados), el sistema debe ejecutar de forma secuencial una serie de *Transacciones Compensatorias*. Estas transacciones de compensación actúan en reversa para deshacer los cambios previos, aplicando un borrado lógico o un cambio de estado a **RECHAZADO**", restableciendo la consistencia del ecosistema bajo el principio de consistencia eventual.

7.2. Complejidad Operativa y Trazabilidad Distribuida

Otro desafío crítico es la pérdida de visibilidad lineal de los flujos de ejecución. En un monolito o sistema síncrono, un error produce un *stack trace* unificado y secuencial. En una arquitectura asíncrona basada en EDA, un evento viaja a través de múltiples hilos, procesos y nodos físicos de red de manera desordenada.

Para solucionar este problema de observabilidad, el equipo debe integrar patrones de **Trazabilidad Distribuida** (*Distributed Tracing*) mediante herramientas basadas en los estándares de OpenTelemetry. Cada petición inicial iniciada por el cliente debe inyectar un identificador global único e inmutable en las cabeceras del mensaje, denominado *Correlation ID* o ID de Correlación. Este identificador se propaga obligatoriamente en cada evento publicado en el Broker y en cada transacción secundaria de los microservicios. De esta manera, el personal de soporte técnico de la infraestructura puede rastrear de forma unificada el ciclo de vida completo de un incidente a través de los logs correlacionados de todos los componentes distribuidos implicados.

8. Conclusiones

Tras el desarrollo del análisis comparativo y su correspondiente aplicación al *Sistema de Gestión de Solicitudes para Soporte Técnico de Internet*, se desprenden las siguientes conclusiones técnicas:

- La arquitectura de microservicios síncronos basada en Petición-Respuesta exhibe debilidades estructurales críticas ante escenarios de alta concurrencia y fallas parciales de infraestructura, debido al acoplamiento temporal estricto que introduce el uso exclusivo de protocolos como HTTP o gRPC tradicionales.
- La Arquitectura Dirigida por Eventos (EDA) proporciona un mecanismo de desacoplamiento superior que incrementa la disponibilidad global del ecosistema, transformando el sistema en un modelo tolerante a fallos de tipo AP, donde la persistencia del estado de negocio se gobierna bajo los principios de la consistencia eventual y el modelo BASE.
- Para el dominio del proyecto integrador propuesto, la solución óptima no radica en la adopción purista de un único estilo arquitectónico, sino en un enfoque híbrido pragmático: utilizar microservicios síncronos asegurados por tokens criptográficos para flujos que demandan respuesta inmediata (como la Autenticación de usuarios), e implementar coreografías asíncronas intermediadas por un *Message Broker* para los procesos intensivos de Gestión de Reportes, Inteligencia Artificial y Notificaciones Multicanal.

9. Referencias Bibliográficas

Referencias

- [1] L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice*, 4th ed. Boston, MA: Addison-Wesley Professional, 2021.
- [2] E. Brewer, “CAP twelve years later: How the ‘rules’ have changed,” *Computer*, vol. 45, no. 2, pp. 23–29, Feb. 2012. [Online]. Available: <https://doi.org/10.1109/MC.2012.37>
- [3] B. Burns, *Designing Distributed Systems: Patterns and Paradigms for Scalable, Reliable Services*, 1st ed. Sebastopol, CA: O’Reilly Media, 2018.
- [4] M. Kleppmann, *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*, 1st ed. Sebastopol, CA: O’Reilly Media, 2017.
- [5] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, 2nd ed. Sebastopol, CA: O’Reilly Media, 2021.
- [6] C. Richardson, *Microservices Patterns: With examples in Java*, 1st ed. Shelter Island, NY: Manning Publications, 2018.
- [7] M. Richards and N. Ford, *Fundamentals of Software Architecture: An Engineering Approach*, 1st ed. Sebastopol, CA: O’Reilly Media, 2020.
- [8] OpenTelemetry Authors, “OpenTelemetry Specifications and Distributed Tracing Standards,” Cloud Native Computing Foundation (CNCF), Tech. Rep., Jan. 2025. [Online]. Available: <https://opentelemetry.io/docs/>